

Linux 复习纲要

基础

1. Linux 操作系统的特点?

- (1) 低成本开发系统
- (2) 可应用于多种硬件平台
- (3) 可定制的内核
- (4) 性能优异
- (5) 良好的网络支持.

2. 常见的命令

- | | | |
|-------------|----------|-----------------|
| (1) 创建文件 | touch | / vi/vim / echo |
| (2) 创建用户 | useradd | |
| (3) 创建组 | groupadd | |
| (4) 授权 | chmod | . chown |
| (5) 用户切换 | su | |
| (6) 创建目录 | mkdir | |
| (7) 修改组(用户) | groupmod | (usermod) |
| (8) 查看 | cat | |
| (9) 用户删除 | userdel | |
| (10) 返回 | cd/ | |

命令	命令含义	格式
useradd	添加用户账号	useradd [选项] 用户名
usermod	设置用户账号属性	usermod [选项] 属性值
userdel	删除对应用户账号	userdel [选项] 用户名
groupadd	添加组账号	groupadd [选项] 组账号
groupmod	设置组账号属性	groupmod [选项] 属性值
groupdel	删除对应组账号	groupdel [选项] 组账号
passwd	设置账号密码	passwd [对应账号]
id	显示用户 ID、组 ID 和用户所属的组列表	id [用户名]
groups	显示用户所属的组	groups [组账号]
who	显示登录到系统的所有用户	who

3. Vim 编辑器

(1) 三种模式

命令模式: 不能编辑文件, 可以有快捷键操作, [默认进入]

编辑模式: 可编辑文件内容 (insert)

末行模式: 在末行输入命令对文件进行操作

(2) 退出方式

:q 退出

:wq! 保存退出

(3) 创建: vi student.txt

(4) insert: 处于插入状态, 可写代码

4. gcc命令

- (1) gcc -E 文件名.c (预编译, 输出预编译结果)
- (2) gcc -S 文件名.c (编译, 预编译得到代码转换得到汇编文件)
- (3) gcc -o 文件名.c (预处理、汇编、编译并链接生成可执行文件)

5. 文件相关命令

- (1) cd - 改变当前工作目录

"cd -" 返回上次工作目录

"." 当前目录 ".." 上级目录

- (2) ls - 列出目录和文件信息

"ls -l" 显示出详细信息

"ls -a" 显示所有 "." 开头的隐藏文件

- (3) mkdir - 创建目录

"mkdir -m" 对新建目录设置存取权限

"mkdir -p" 一次性创建多个目录

- (4) cat - 连接并显示指定的一个或多个文件的有关信息.

"cat -n" 由第一行开始对所有输出的行数编号

"cat -p" 类似 -n, 对于空行不编号

- (5) cp, mv, rm

a. cp - 复制

- | | |
|----|-------------------------|
| -a | 保留链接、文件属性、复制子目录 |
| -d | 复制保留链接 |
| -f | 删除不存在目标文件不提示. |
| -i | 交互式复制 (需确认) |
| -p | 除源文件内容外, 把其修改时间、访问权限也复制 |
| -r | 递归复制该目录下所有子目录文件. |

b. mv - 改名或转换文件目录 { -i 避免误复盖文件

c. rm - 删除 { -i 进行强制删除 -f 禁止交互操作.
-f 忽略不存在的文件.
-r 将参数中目录和子目录递归删除.

16) chown - 修改文件所有者、组别

chgrp - 改变文件的组所有权.

chmod - 改变文件的访问权限

grep - 搜索指定文件特定内容, 并标准输出

find - 指定目录中搜索文件(所有用户), 可以混合查找

6. 文件或命令的操作权限

1) 读(r), 写(w), 执行(x)

权限: 4 2 1

rwx - 本人权限

r-- - 所在组人员权限

rw- - 其它人员权限.

例① rw-r--rwx 权限描述为 647 转为二进制描述 110 100 111
6 4 7

② r--r--r-- 444 100 100 100

③ rwxr--r-- 744 111 100 100

7. Linux 常见目录的含义及作用?

/boot 在Linux启动所需的内核文件和镜像文件

/root 超级用户的主目录

/home 用户的主目录, 每个用户都有一个自己的目录

/var 动态数据保存位置.

Linux 文件系统目录结构	
目 录	目 录 内 容
/bin	bin 就是二进制 (binary) 的英文缩写。在这里存放 Linux 常用操作命令的执行文件, 如 mv、ls、mkdir 等。有时, 这个目录的内容和/usr/bin 里面的内容一样, 它们都是放置一般用户使用的执行文件
/boot	这个目录下存放操作系统启动时所要用的程序。如启动 grub 就会用到其下的/boot/grub 子目录
/dev	该目录中包含了所有 Linux 系统中使用的外部设备。要注意的是, 这里并不是存放外部设备的驱动程序, 它实际上是一个访问这些外部设备的端口。由于在 Linux 中, 所有的设备被当作文件进行操作, 比如: /dev/cdrom 代表光驱, 用户可以非常方便地像访问文件、目录一样对其进行访问
/etc	该目录下存放了系统管理时要用到的各种配置文件和子目录。如网络配置文件、文件系统、x 系统配置文件、设备配置信息、设置用户信息等都在这个目录下。系统在启动过程中需要读取其参数并进行相应的配置
/etc/rc.d	该目录主要存放 Linux 启动和关闭时要用到的脚本文件, 在后面的启动详解中还会进一步地讲解
/etc/rc.d/init	该目录存放所有 Linux 服务默认的启动脚本 (在新版本的 Linux 中还用到/etc/xinetd.d 目录下的内容)
/home	该目录是 Linux 系统中默认的用户工作根目录。如前面在 1.3.1 节中所述, 执行 adduser 命令后系统会在/home 目录下为对应账号建立一个同名的主目录
/lib	该目录是用来存放系统动态链接共享库的。几乎所有的应用程序都会用到这个目录下的共享库。因此, 千万不要轻易对这个目录进行操作
/lost+found	该目录在大多数情况下都是空的。只有当系统产生异常时, 会将一些遗失的片段放在此目录下
/media	该目录下是光驱和软驱的挂载点, Fedora Core 4 已经可以自动挂载光驱和软驱
/misc	该目录下存放从 DOS 下进行安装的实用工具, 一般为空
/mnt	该目录是软驱、光驱、硬盘的挂载点, 也可以临时将别的文件系统挂载到此目录下
/proc	该目录是用于放置系统核心与执行程序所需的一些信息。而这些信息是在内存中由系统产生的, 故不占用硬盘空间
/root	该目录是超级用户登录时的主目录
/sbin	该目录用来存放系统管理员的常用的系统管理程序
/tmp	该目录用来存放不同程序执行时产生的临时文件。一般 Linux 安装软件的默认安装路径就是这里
/usr	这是一个非常重要的目录, 用户的很多应用程序和文件都存放在这个目录下, 类似于 Windows 下的 Program Files 的目录
/usr/bin	系统用户使用的应用程序
/usr/sbin	超级用户使用的比较高级的管理程序和系统守护程序
/usr/src	内核源代码默认的放置目录
/srv	该目录存放一些服务启动之后需要提取的数据
/sys	这是 Linux 2.6 内核的一个很大的变化。该目录下安装了 2.6 内核中新出现的一个文件系统 sysfs。sysfs 文件系统集成了下面 3 种文件系统的信息: 针对进程信息的 proc 文件系统、针对设备的 devfs 文件系统以及针对伪终端的 devpts 文件系统。该文件系统是内核设备树的一个直观反映。当一个内核对象被创建的时候, 对应的文件和目录也在内核对象子系统中被创建
/var	这也是一个非常重要的目录, 很多服务的日志信息都存放在这里

一. 标准 I/O

1. OS 提供的功能有哪些？

用户程序向操作系统提出请求的接口就是系统调用。

这些系统调用按照功能大致分为：进程控制、进程间通信、文件系统控制、存储管理、网络管理、套接字控制、用户管理等。

2. Linux 遵循 POSIX 标准

POSIX 标准是由 IEEE 和 ISO/IEC 共同开发的标准系统，该标准基于当时的 UNIX 实践和经验，描述了操作系统的系统调用编程接口（实际上就是 API），用于保证应用程序可以在源代码一级上在多种操作系统上移植运行。

3. 标准 I/O 遵循 ANSI 规范

4. 标准 I/O 流的三种缓冲类型 P4

标准 I/O 中的流的缓冲类型有以下 3 种。

(1) 全缓冲。在这种情况下，当填满标准 I/O 缓冲区后才进行实际 I/O 操作。对于存放在磁盘上的普通文件，用标准 I/O 打开时默认是全缓冲的。当缓冲区已满或执行 flush 操作时才会进行磁盘操作。

(2) 行缓冲。在这种情况下，当在输入和输出中遇到换行符时执行 I/O 操作。标准输入流和标准输出流就是使用行缓冲的典型例子。

(3) 无缓冲。不对 I/O 操作进行缓冲，即在对流的读写时会立刻操作实际的文件。标准出错流是不带缓冲的，这就使得出错信息可以立刻显示在终端上，而不管输出的内容是否包含换行符。

5. fopen 函数中参数 mode 的取值类型 P6 表 1.2

表 1.2 mode 取值说明

r 或 rb	打开只读文件，该文件必须存在
r+ 或 r+b	打开可读写的文件，该文件必须存在
w 或 wb	打开只写文件，若文件存在则文件长度为 0，即会擦写文件以前的内容；若文件不存在则建立该文件
w+ 或 w+b	打开可读写文件，若文件存在则文件长度为 0，即会擦写文件以前的内容；若文件不存在则建立该文件
a 或 ab	以附加的方式打开只写文件。若文件不存在，则会建立该文件；如果文件存在，写入的数据会被加到文件尾，即文件原先的内容会被保留
a+ 或 a+b	以附加方式打开可读写的文件。若文件不存在，则会建立该文件；如果文件存在，写入的数据会被加到文件尾，即文件原先的内容会被保留

6. 文件复制的功能代码 (fread与fwrite函数的用法) P14.

```
/*mycopy.c*/
#include <stdio.h>
#include <errno.h>
#define N 64
int main (int argc, char *argv[])
{
    int n;
    char buf[N];
    FILE *fps, *fpd;

    if (argc < 3)
    {
        printf ( " Usage : %s <src_file> <dst_file>\n" , argv[0] );
        return -1;
    }

    if ( ( fps = fopen ( argv[1], " r " ) ) == NULL )
    {
        fprintf ( stderr, " fail to fopen %s : %s\n" , argv[1], strerror ( errno ) );
        return -1;
    }

    if ( ( fpd = fopen ( argv[2], " w " ) ) == NULL )
    {
        fprintf ( stderr, " fail to fopen %s : %s\n" , argv[2], strerror ( errno ) );
        fclose ( fps );
        return -1;
    }

    while ( ( n = fread ( buf, 1, N, fps ) ) >= 0 )
    {
        fwrite ( buf, 1, N, fpd );
    }

    fclose ( fps );
    fclose ( fpd );
    return 0;
}
```

表 1.10 fread()函数语法要点

所需头文件	#include <stdio.h>
函数原型	size_t fread (void * ptr, size_t size, size_t nmemb, FILE * stream);
函数参数	ptr: 存放读入记录的缓冲区
	size: 读取的每个记录的大小
	nmemb: 读取的记录数
	stream: 要读取的文件流
函数返回值	成功: 返回实际读取到的 nmemb 数目
	失败: EOF

fwrite()函数语法要点如表 1.11 所示。

表 1.11 fwrite()函数语法要点

所需头文件	#include <stdio.h>
函数原型	size_t fwrite (const void * ptr, size_t size, size_t nmemb, FILE * stream);
函数参数	ptr: 存放写入记录的缓冲区
	size: 写入的每个记录的大小
	nmemb: 写入的记录数
	stream: 要写入的文件流
函数返回值	成功: 返回实际写入的 nmemb 数目
	失败: EOF

二. 文件 I/O

1. VFS 的功能描述 (P18-19)

- ① 对具体文件系统的数据结构进行抽象, 以一种统一的数据接口进行管理
- ② 接受上层系统的调用
- ③ 支持多种具体文件系统之间的相互访问, 接受内核其他系统的操作请求

2. 文件的 6 种类型:

- 普通文件
- 目录文件
- 符号链接文件
- 管道文件
- 套接字文件
- 设备文件

3. 文件 I/O 与标准 I/O 的区别

文件 I/O 遵循 POSIX 相关标准, 标准 I/O 遵循 ANSI C 相关标准。

文件 I/O 每次操作都会进行相关系统调用, 标准 I/O 会先读写缓冲区。

文件 I/O 用文件描述符打开文件, 标准 I/O 用 FILE。
前者

4. 文件 I/O 操作的 open 函数的参数 O_RDONLY 含义

O_RDONLY 只读 O_WRONLY 只写 O_RDWR 读写 O_CREAT 不存在则创建新文件
(read) 打开文件 (write)

O_EXCL 测试文件是否存在 O_NOCTTY 若打开的是终端文件, 那么该终端不会成为当前进程的控制终端
O_TRUNC 若文件存在, 则删除所有数据

O_APPEND: 添加方式打开文件
(从末尾开始写)

5. fcntl 函数的作用

1. 管理文件锁 (记录锁)
2. 获取/设置文件相关标志位
3. 复制文件描述符

6. 文件锁 = 建议性锁和强制性锁 2 种, 区别?

建议性锁: 访问文件前检查是否存在锁, 尊重已有锁

强制性锁: 文件上锁, 内核将阻止其他任何程序对该文件的读写操作

7. 实验代码阅读 (P33-40)

1. 任务与进程的关系

三、进程控制

① 通常一个任务是一个程序的一次运行，一个任务包含一个或多个完成独立功能的子任务，这个独立的子任务是进程或者是线程。

② 一次任务的运行可以激活多个进程，这些进程相互合作来完成该任务的最终目标。

2. 进程的特性。

进程具有并发性、动态性、交互性和独立性等主要特性。

(1) **并发性**。指的是系统中多个进程可以同时并发执行，相互之间不受干扰。

(2) **动态性**。指的是进程都有完整的生命周期，而且在进程的生命周期内，进程的状态是不断变化的，另外，进程具有动态的地址空间（包括代码、数据和进程控制块等）。

(3) **交互性**。指的是进程在执行过程中可能会与其他进程发生直接和间接的通信，如进程同步和进程互斥等，需要为此添加一定的进程处理机制。

(4) **独立性**。指的是进程是一个相对完整的资源分配和调度的基本单位，各个进程的地址空间是相互独立的，只有采用某些特定的通信机制才能实现进程之间的通信。

干扰
内存地址空间
数据地址
地址空间

3. 进程与线程的关系

① 线程可以对进程的内存空间和资源进行访问，并与同一进程中的其他线程交互。

② **SO**。线程的上下文切换的开销比进程小得多。

③ 一个进程可以拥有多个线程，其中每个线程共享该进程所拥有的资源。

⚠ (因为线程共享了进程的资源和地址空间，因此任何对系统资源的操作都会给其他线程带来影响)

4. 线程的类型 交互式、批处理、守护。

5. 进程信息由 task_struct 数据结构描述。

23:51 12月5日周一

...

57%



2. Linux 下的进程结构

进程不但包括程序的指令和数据，而且包括程序计数器和处理器的所有寄存器以及存储临时数据的进程堆栈。

因为 Linux 是一个多任务的操作系统，所以其他的进程必须等到操作系统将处理器使用权分配给自己之后才能运行。当正在运行的进程需要等待其他的系统资源时，Linux 内核将取得处理器的控制权，按照某种调度算法将

处理器分配给某个正在等待执行的进程。

内核将所有进程存放在双向循环链表（进程链表）中，链表的每一项都是 `task_struct`，称为进程控制块的结构。该结构包含了与一个进程相关的所有信息，在 `<include/Linux/sched.h>` 文件中定义。`task_struct` 内核结构比较大，它能完整地描述一个进程，如进程的状态、进程的基本信息、进程标识符、内存相关信息、父进程相关信息、与进程相关的终端信息、当前工作目录、打开的文件信息、所接收的信号信息等。



进程的状态

下面详细讲解 `task_struct` 结构中最为重要的两个域：`state`（进程状态）和 `pid`（进程标识符）。

(1) 进程状态。

6. 进程的运行状态

Linux 中的进程有以下几种主要状态。

① 运行状态（`TASK_RUNNING`）。进程当前正在运行，或者正在运行队列中等待调度。

② 可中断的阻塞状态（`TASK_INTERRUPTIBLE`）。进程处于阻塞（睡眠）状态，正在等待某些事件发生或能够占用某些资源。处在这种状态下的进程可以被信号中断。接收到信号或被显式地唤醒呼叫（如调用 `wake_up` 系列宏：`wake_up`、`wake_up_interruptible` 等）唤醒之后，进程将转变为 `TASK_RUNNING` 状态。

③ 不可中断的阻塞状态（`TASK_UNINTERRUPTIBLE`）。此进程状态类似于可中断的阻塞状态（`TASK_INTERRUPTIBLE`），只是它不会处理信号，把信号传递到这种状态下的进程不能改变它的状态。在一些特定的情况下（进程必须等待，直到某些不能被中断的事件发生），这种状态是很有用的。只有在它所等待的事件发生时，进程才被显式地唤醒呼叫唤醒。

7. 进程描述符: 进程号 (PID)、父进程号 (PPID)

8. fork 函数作用.

(1) 进程的创建和执行。

许多操作系统都提供产生进程的机制，也就是首先在新的地址空间里创建进程、读入可执行文件，最后再开始执行。Linux 中进程的创建很特别，它把上述步骤分解到两个单独的函数中执行：fork() 和 exec 函数族。首先，

46

第3章
Linux 多任务编程

fork 函数作用.



注意父子进程的区别

fork() 通过复制当前进程创建一个子进程，子进程与父进程的区别仅仅在于不同的 PID、PPID 和某些资源及统计量。exec 函数族负责读取可执行文件并将其载入地址空间开始运行。

⚠️ 要注意的是，Linux 中的 fork() 使用的是写时复制 (copy on write) 的技术，也就是内核在创建进程时，其资源并没有立即被复制过来，而是被推迟到需要写入数据的时候才发生。在此之前只是以只读的方式共享父进程的资源。写时复制技术可以使 Linux 拥有快速执行的能力，因此这个优化是非常重要的。



9.

fork()函数的简单的示例程序如下。

```
int main ( void )
{
    pid_t ret;

    /*调用fork()函数*/
    ret = fork();

    /*通过ret的值来判断fork()函数的返回情况，首先进行出错处理*/
    if ( ret == -1 )
    {
        perror ( "fork error" );
        return -1;
    }
    else if ( ret == 0 ) /*返回值为0代表子进程*/
    {
        printf ( "In child process!! ret is %d, My PID is %d\n", ret, getpid() );
    }
    else /*返回值大于0代表父进程*/
    {
        printf ( "In parent process!! ret is %d, My PID is %d\n", ret, getpid() );
    }

    return 0;
}
```

编译并执行程序，结果如下。

```
$ gcc fork.c -o fork -Wall
```

```
./fork
```

```
In parent process!! ret is 3876, My PID is 3875
```

```
In child process!! ret is 0, My PID is 3876
```

从该示例中可以看出，使用 fork()函数新建了一个子进程，其中的父进程返回子进程的进程号，而子进程的返回值为 0。



getpid()获得当前进程的进程号。



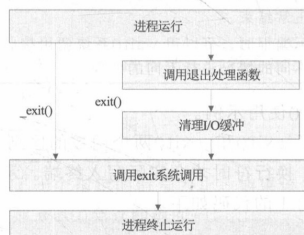


图 3.5 exit()和 _exit()函数流程图

从图 3.5 中可以看出，`_exit()`函数的作用是直接使进程停止运行，清除其使用的内存空间，并清除其在内核中的各种数据结构；`exit()`函数则在这些基础上做了一些包装，在执行退出之前加了若干道工序。`exit()`函数与 `_exit()`函数最大的区别就在于 `exit()`函数在终止当前进程之前要检查该进程打开了哪些文件，并把文件缓冲区中的内容写回文件，就是图中的“清理 I/O 缓冲”一项。

10. `exit()`与`_exit()`区别

4. `wait()`和 `waitpid()`

(1) `wait()`和 `waitpid()`函数说明。

`wait()`函数用于使父进程（也就是调用 `wait()`的进程）阻塞，直到一个子进程结束或者该进程收到了一个指定的信号为止。如果该父进程没有子进程或者其他的子进程已经结束，则 `wait()`会立即返回-1。

`waitpid()`的作用和 `wait()`一样，但它并不一定等待第一个终止的子进程。`waitpid()`有若干选项，可提供一个非阻塞版本的 `wait()`功能。实际上 `wait()`函数只是 `waitpid()`函数的一个特例，在 Linux 内部实现 `wait()`函数时直接调用的就是 `waitpid()`函数。

(2) `wait()`和 `waitpid()`函数格式说明。

11. `wait()`和`waitpid()`区别

12. 看懂代码 4.1. 71-75.

四. 进程通信

1、进程间通信的 6 种方式? p78-79

- (1) 无名管道及有名管道。
- (2) 信号。
- (3) 消息队列。
- (4) 共享内存。
- (5) 信号量。
- (6) 套接字。

(1) 无名管道 (pipe) 及有名管道 (fifo)。无名管道可用于具有亲缘关系进程间的通信 (例如, fork() 函数建立父子进程通信); 有名管道除具有和无名管道相似的功能外, 它还允许无亲缘关系进程使用。

(2) 信号 (signal)。信号是在软件层次上对中断机制的一种模拟, 它是比较复杂的通信方式, 用于通知进程某事件发生。一个进程收到一个信号与处理器收到一个中断请求处理的过程类似。

(3) 消息队列 (message queue)。消息队列是消息的链接表, 包括 POSIX 消息队列和 System V 消息队列。它克服了前两种通信方式中信息量有限的缺点, 具有写权限的进程可以按照一定的规则向消息队列中添加新消息; 对消息队列有读权限的进程则可以从消息队列中读取消息。

(4) 共享内存 (shared memory)。可以说这是最有效的进程间通信方

式。它使得多个进程可以访问同一块内存空间, 不同进程可以及时看到对方进程中对共享内存中数据的更新。这种通信方式需要依靠某种同步机制, 如互斥锁和信号量等。

(5) 信号量 (semaphore)。主要作为进程之间以及同一进程的不同线程之间的同步和互斥手段。

(6) 套接字 (socket)。这是一种使用更广泛的进程间通信机制, 它可用于网络中不同主机之间的进程间通信, 应用非常广泛。

2、无名管道（单工）与有名管道的区别？

(1) 无名管道 (pipe) 及有名管道 (fifo)。无名管道可用于具有亲缘关系进程间的通信 (例如, fork() 函数建立父子进程通信); 有名管道除具有和无名管道相似的功能外, 它还允许无亲缘关系进程使用。

1. 无名管道

无名管道是 Linux 中管道通信的一种原始方法, 如图 4.2 (a) 所示, 它具有如下特点。

(1) 只能用于具有亲缘关系的进程之间的通信 (也就是父子进程或者兄弟进程之间)。

(2) 是一个单工的通信模式, 具有固定的读端和写端。

(3) 管道也可以看成一种特殊的文件, 对于它的读写也可以使用普通的 read()、write() 等函数。但是它不属于任何文件系统, 并且只存在于内存中。

2. 有名管道

有名管道 (FIFO) 是对无名管道的一种改进, 如图 4.2 (b) 所示, 它具有如下特点。

(1) 它可以使互不相关的两个进程实现彼此通信。

(2) 该管道可以通过路径名来指出, 并且在文件系统中是可见的。在建立了管道之后, 两个进程就可以把它当作普通文件进行读写操作, 使用非常方便。

(3) FIFO 严格地遵循先进先出规则, 对管道及 FIFO 的读总是从开始处返回数据, 对它们的写则把数据添加到末尾。有名管道不支持如 lseek() 等文件定位操作。

3、有名管道使用 mkfifo() 函数, 类似命令行的 mknod, 可以使用文件 I/O 的系列函数

有名管道 (FIFO) 的创建可以使用 mkfifo() 函数, 该函数类似文件中的 open() 操作, 可以指定管道的路径和访问权限 (用户也可以在命令行使用 “mknod <管道名>” 来创建有名管道)。

在创建管道成功之后, 就可以使用 open()、read() 和 write() 这些函数了。与普通文件一样, 对于为读而打开的管道可在 open() 中设置 O_RDONLY, 对于为写而打开的管道可在 open() 中设置 O_WRONLY。

4、信号通信机制--可用于异步通信

5、常用的信号处理函数: kill()、raise()、alarm()、signal()、sigaction()、pause() 等

6、常用的信号含义：SIGINT、SIGQUIT、SIGKILL 等,p88 表 4.4

表 4.4 常见信号的含义及其默认操作

信号名	含义	默认操作
SIGHUP	该信号在用户终端连接(正常或非正常)结束时发出,通常是在终端的控制进程结束时,通知同一会话内的各个进程与控制终端不再关联	终止进程
<u>SIGINT</u>	该信号在用户输入 INTR 字符(通常是 Ctrl+C)时发出,终端驱动程序发送此信号并送到前台进程中的每一个进程	终止进程
<u>SIGQUIT</u>	该信号和 SIGINT 类似,但由 QUIT 字符(通常是 Ctrl+\)来控制	终止进程
SIGILL	该信号在一个进程企图执行一条非法指令时(可执行文件本身出现错误,或者试图执行数据段、堆栈溢出时)发出	终止进程
SIGFPE	该信号在发生致命的算术运算错误时发出,这里不仅包括浮点运算错误,还包括溢出及除数为 0 等其他所有的算术的错误	终止进程
<u>SIGKILL</u>	该信号用来立即结束程序的运行,并且不能被阻塞、处理和忽略	终止进程
SIGALARM	该信号当一个定时器到时的时候发出	终止进程
SIGSTOP	该信号用于暂停一个进程,且不能被阻塞、处理或忽略	暂停进程
SIGTSTP	该信号用于交互停止进程,用户在输入 SUSP 字符时(通常是 Ctrl+Z)发出这个信号	停止进程
SIGCHLD	子进程改变状态时,父进程会收到这个信号	忽略

7、信号量可用于进程的同步与互斥

8. PV 操作的含义? p96

PV 原子操作的具体定义如下。

(1) P 操作。如果有可用的资源(信号量值大于 0),则占用一个资源(信号量值减 1,进入临界区代码);如果没有可用的资源(信号量值等于 0),则被阻塞,直到系统将资源分配给该任务(进入等待队列,一直等到有资源时被唤醒)。

(2) V 操作:如果在该信号量的等待队列中有任务在等待资源,则唤醒一个阻塞任务。如果没有任务等待它,则释放一个资源(信号量值加 1)。

9. 常见的函数:semget(), semctl(), semop()

表 4.12 semget()函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h></pre>
函数原型	<pre>int semget(key_t key, int nsems, int semflg);</pre>
函数传入值	key: 信号量的键值,多个进程可以通过它访问同一个信号量。其中有个特殊值 IPC_PRIVATE,用于创建当前进程的私有信号量
	nsems: 需要创建的信号量数目,通常取值为 1
函数返回值	semflg: 同 open()函数的权限位,也可以用八进制表示法,其中使用 IPC_CREAT 标志创建新的信号量,即使该信号量已经存在(具有同一个键值的信号量已在系统中存在),也不会出错。如果同时使用 IPC_EXCL 标志可以创建一个新的唯一的信号量,此时如果该信号量已经存在,该函数会返回出错
	成功: 信号量标识符,在信号量的其他函数中都会使用该值 出错: -1

表 4.13 semctl()函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h></pre>
函数原型	int semctl(int semid, int semnum, int cmd, union semun arg);
函数传入值	semid: semget()函数返回的信号量标识符
	semnum: 信号量编号, 当使用信号量集时才会被用到。通常取值为 0, 就是使用单个信号量 (也是第一个信号量)
	cmd: 指定对信号量的各种操作, 当使用单个信号量 (而不是信号量集) 时, 常用的操作有以下几种。 IPC_STAT: 获得该信号量 (或者信号量集合) 的 semid_ds 结构, 并存放在由第四个参数 arg 结构变量的 buf 域指向的 semid_ds 结构中。semid_ds 是在系统中描述信号量的数据结构 IPC_SETVAL: 将信号量值设置为 arg 的 val 值 IPC_GETVAL: 返回信号量的当前值 IPC_RMID: 从系统中删除信号量 (或者信号量集)
函数返回值	arg: 是 union semun 结构, 可能在某些系统中不给出该结构的定义, 此时必须由程序员自己定义。 <pre>union semun { int val; struct semid_ds *buf; unsigned short *array; }</pre> 成功: 根据 cmd 值的不同而返回不同的值 IPC_STAT、IPC_SETVAL、IPC_RMID: 返回 0 IPC_GETVAL: 返回信号量的当前值 出错: -1

表 4.14 semop()函数语法要点

所需头文件	<pre>#include <sys/types.h> #include <sys/ipc.h> #include <sys/sem.h></pre>
函数原型	int semop(int semid, struct sembuf *sops, size_t nsops)
函数传入值	semid: semget()函数返回的信号量标识符

函数传入值	sops: 指向信号量操作数组, 一个数组包括以下成员。 <pre> struct sembuf { short sem_num; /* 信号量编号, 使用单个信号量时, 通常取值为 0 */ short sem_op; /* 信号量操作: 取值为-1 则表示 P 操作, 取值为+1 则表示 V 操作*/ short sem_flg; /* 通常设置为 SEM_UNDO。这样在进程没释放信号量而退出时, 系统自动释放该进程中未释放的信号量 */ } </pre>
	nsops: 操作数组 sops 中的操作个数 (元素数目), 通常取值为 1 (一个操作)
函数返回值	成功: 信号量标识符, 在信号量的其他函数中都会使用该值
	出错: -1

10. 代码熟悉:p99-102

```

/* sem_com.c */
#include "sem_com.h"

/* 信号量初始化 (赋值) 函数*/
int init_sem(int sem_id, int init_value)
{
    union semun sem_union;

    sem_union.val = init_value; /* init_value为初始值 */
    if (semctl(sem_id, 0, SETVAL, sem_union) == -1)
    {
        perror("Initialize semaphore");
        return -1;
    }

    return 0;
}

/* 从系统中删除信号量的函数 */

```

```
int del_sem(int sem_id)
```

```
{  
    union semun sem_union;  
    if (semctl(sem_id, 0, IPC_RMID, sem_union) == -1)  
    {  
        perror("Delete semaphore");  
        return -1;  
    }  
}
```

```
/* P操作函数 */
```

```
int sem_p(int sem_id)
```

```
{  
    struct sembuf sem_b;  
    sem_b.sem_num = 0; /* 单个信号量的编号应该为0 */  
    sem_b.sem_op = -1; /* 表示P操作 */  
    sem_b.sem_flg = SEM_UNDO; /* 系统自动释放将会在系统中残留的信号量 */  
    if (semop(sem_id, &sem_b, 1) == -1)  
    {  
        perror("P operation");  
        return -1;  
    }  
    return 0;  
}
```

```
/* V操作函数 */
```

```
int sem_v(int sem_id)
```

```
{  
    struct sembuf sem_b;  
    sem_b.sem_num = 0; /* 单个信号量的编号应该为0 */  
    sem_b.sem_op = 1; /* 表示V操作 */  
    sem_b.sem_flg = SEM_UNDO; /* 系统自动释放将会在系统中残留的信号量 */  
    if (semop(sem_id, &sem_b, 1) == -1)  
    {  
        perror("V operation");  
        return -1;  
    }  
}
```



```
return 0;
```

```
}
```

以下示例说明信号量的概念以及基本用法。在示例程序中，首先创建一个子进程，接下来使用信号量来控制两个进程（父子进程）之间的执行顺序。

```
/* fork.c */  
  
#include <sys/types.h>  
#include <unistd.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <sys/types.h>  
#include <sys/ipc.h>  
#include <sys/shm.h>  
#define DELAY_TIME 3 /* 为了突出演示效果，等待几秒钟*/  
  
int main(void)  
{  
    pid_t result;  
    int sem_id;  
  
    sem_id = semget(ftok(".", 'a'), 1, 0666|IPC_CREAT); /* 创建一个信号量*/  
    init_sem(sem_id, 0);  
  
    /*调用fork()函数*/  
    result = fork();  
    if(result == -1)  
    {  
        perror("Fork\n");  
    }  
    else if (result == 0) /*返回值为0代表子进程*/  
    {  
        printf("Child process will wait for some seconds...\n");  
        sleep(DELAY_TIME);  
        printf("The returned value is %d in the child process(PID = %d)\n",  
            result, getpid());  
        sem_v(sem_id);  
    }  
}
```

```

    }
    else /*返回值大于0代表父进程*/
    {
        sem_p(sem_id);
        printf("The returned value is %d in the father process(PID = %d)\n",
            result, getpid());
        sem_v(sem_id);
        del_sem(sem_id);
    }
    exit(0);
}

```

读者可以先从该程序中删除信号量相关的代码部分并观察运行结果。

```

$ ./simple_fork
Child process will wait for some seconds... /*子进程在运行中*/
The returned value is 4185 in the father process(PID = 4184) /*父进程先结束*/
[...]$ The returned value is 0 in the child process(PID = 4185) /*子进程后结束*/

```

再添加信号量的控制部分并运行结果。

```

$ ./sem_fork
Child process will wait for some seconds...
/*子进程在运行中，父进程在等待子进程结束*/
The returned value is 0 in the child process(PID = 4185) /*子进程结束了*/
The returned value is 4185 in the father process(PID = 4184) /*父进程结束*/

```

本示例说明使用信号量怎么解决多进程之间存在的同步问题。在后面讲述的共享内存和消息队列的示例中，会看到使用信号量实现多进程之间的互斥。

11、共享内存——效率最高的一种方式

12、几个函数 shmget()、shmat()、shmctl()、shmdt()等作用？

表 4.15 shmget()函数语法要点

所需头文件	#include <sys/types.h> #include <sys/ipc.h> #include <sys/shm.h>
函数原型	int shmget(key_t key, int size, int shmflg);
函数传入值	key: 共享内存的键值，多个进程可以通过它访问同一个共享内存。其中有个特殊值 IPC_PRIVATE，用于创建当前进程的私有共享内存
	size: 共享内存区大小
	shmflg: 同 open()函数的权限位，也可以用八进制表示法
函数返回值	成功: 共享内存段标识符
	出错: -1

表 4.16 shmat()函数语法要点

所需头文件	#include <sys/types.h> #include <sys/ipc.h> #include <sys/shm.h>
函数原型	char *shmat(int shmid, const void *shmaddr, int shmflg);
函数传入值	shmid: 要映射的共享内存区标识符
	shmaddr: 将共享内存映射到指定地址（若为 0 则表示系统自动分配地址并把该段共享内存映射到调用进程的地址空间）

函数传入值	shmflg	SHM_RDONLY: 共享内存只读
		默认 0: 共享内存可读写
函数返回值	成功: 被映射的段地址	
	出错: -1	

表 4.17 shmdt()函数语法要点

所需头文件	#include <sys/types.h> #include <sys/ipc.h> #include <sys/shm.h>
函数原型	int shmdt(const void *shmaddr);
函数传入值	shmaddr: 被映射的共享内存段地址
函数返回值	成功: 0
	出错: -1

13、有亲缘关系的共享内存（p104-108）与没有亲缘关系的共享内存的示例（p122-126）

P104-108 有亲缘关系...

```
/* shmem.c */
```

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define BUFFER_SIZE 2048
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    int shmid;
```

```
    char *shm_addr;
```

```
    char flag[] = "WROTE";
```

```
    char buff[BUFFER_SIZE];
```

```
    /* 创建共享内存 */
```

```
    if ((shmid = shmget(IPC_PRIVATE, BUFFER_SIZE, 0666)) < 0)
```

```
    {
```

```
        perror("shmget");
```

```
        exit(1);
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Create shared-memory > %d\n", shmid);
```

```
    }
```

```
    /* 显示共享内存情况 */
```

```
    system("ipcs -m");
```

```
    pid = fork();
```

```
    if (pid == -1)
```

```
    {
```

```
        perror("fork");
```

```
        exit(1);
```

```
    }
```

```
    else if (pid == 0) /* 子进程处理 */
```

```
    {
```

```
/*映射共享内存*/
```

```
if ((shm_addr = shmat(shmid, 0, 0)) == (void*)-1)
```

```
{
```

```
    perror("Child: shmat");
```

```
    exit(1);
```

```
}
```

```
else
```

```
{
```

```
    printf("Child: Attach shared-memory: %p\n", shm_addr);
```

```
}
```

```
system("ipcs -m");
```

```
/* 通过检查在共享内存的头部是否标志字符串WROTE来确认
```

```
父进程已经向共享内存写入有效数据 */
```

```
while (strcmp(shm_addr, flag, strlen(flag)))
```

```
{
```

```
    printf("Child: Wait for enable data...\n");
```

```
    sleep(5);
```

```
}
```

```
/* 获取共享内存的有效数据并显示 */
```

```
strcpy(buff, shm_addr + strlen(flag));
```

```
printf("Child: Shared-memory : %s\n", buff);
```

```
/* 解除共享内存映射 */
```

```
if ((shmdt(shm_addr)) < 0)
```

```
{
```

```
    perror("shmdt");
```

```
    exit(1);
```

```
}
```

```
else
```

```
{
```

```
    printf("Child: Deattach shared-memory\n");
```

```
}
```

```
system("ipcs -m");
```

```

/* 删除共享内存 */
if (shmctl(shmid, IPC_RMID, NULL) == -1)
{
    perror("Child: shmctl(IPC_RMID)\n");
    exit(1);
}
else
{
    printf("Delete shared-memory\n");
}

system("ipcs -m");
}
else /* 父进程处理 */
{
    /*映射共享内存*/
    if ((shm_addr = shmat(shmid, 0, 0)) == (void*)-1)
    {
        perror("Parent: shmat");
        exit(1);
    }
    else
    {
        printf("Parent: Attach shared-memory: %p\n", shm_addr);
    }

    sleep(1);
    printf("\nInput some string:\n");
    fgets(buff, BUFFER_SIZE, stdin);
    strncpy(shm_addr + strlen(flag), buff, strlen(buff));
    strncpy(shm_addr, flag, strlen(flag));

    /* 解除共享内存映射 */
    if ((shmdt(shm_addr)) < 0)

```



```

    {
        perror("Parent: shmdt");
        exit(1);
    }
    else
    {
        printf("Parent: Deattach shared-memory\n");
    }
    system("ipcs -m");

    waitpid(pid, NULL, 0);
    printf("Finished\n");
}

exit(0);
}

```

P122-126 没有亲缘关系---

共享内存缓冲区的数据结构的定义如下。

```

/* shm_com.h */
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_BUFF_SZ 2048

struct shm_buff
{
    int pid;
    char buffer[SHM_BUFF_SZ];
};

```

“生产者”程序部分如下。

```
/* sem_com.h 和 sem_com.c 与“信号量”小节示例中的同名程序相同 */
/* producer.c */
#include "shm_com.h"
#include "sem_com.h"
#include <signal.h>
int ignore_signal(void)
{ /* 忽略一些信号，免得非法退出程序 */
    signal(SIGINT, SIG_IGN);
    signal(SIGSTOP, SIG_IGN);
    signal(SIGQUIT, SIG_IGN);
    return 0;
}

int main()
{
    void *shared_memory = NULL;
    struct shm_buff *shm_buff_inst;
    char buffer[BUFSIZ];
    int shmid, semid;
    /* 定义信号量，用于实现访问共享内存的进程之间的互斥 */
    ignore_signal(); /* 防止程序非正常退出 */
    semid = semget(ftok(".", 'a'), 1, 0666|IPC_CREAT); /* 创建一个信号量 */
    init_sem(semid); /* 初始值为1 */

    /* 创建共享内存 */
    shmid = shmget(ftok(".", 'b'), sizeof(struct shm_buff), 0666|IPC_CREAT);
```

```

if (shmid == -1)
{
    perror("shmget failed");
    del_sem(semid);
    exit(1);
}

/* 将共享内存地址映射到当前进程地址空间 */
shared_memory = shmat(shmid, (void*)0, 0);
if (shared_memory == (void*)-1)
{
    perror("shmat");
    del_sem(semid);
    exit(1);
}

printf("Memory attached at %X\n", (int)shared_memory);
/* 获得共享内存的映射地址 */
shm_buff_inst = (struct shared_use_st *)shared_memory;
do
{
    sem_p(semid);
    printf("Enter some text to the shared memory(enter 'quit' to exit):");
    /* 向共享内存写入数据 */
    if (fgets(shm_buff_inst->buffer, SHM_BUFF_SZ, stdin) == NULL)
    {
        perror("fgets");
        sem_v(semid);
        break;
    }
    shm_buff_inst->pid = getpid();
    sem_v(semid);
} while(strncmp(shm_buff_inst->buffer, "quit", 4) != 0);

/* 删除信号量 */
del_sem(semid);

```



```

/* 删除共享内存到当前进程地址空间中的映射 */
if (shmdt(shared_memory) == 1)
{
    perror("shmdt");
    exit(1);
}
exit(0);
}

```

“消费者”程序部分如下。

```

/* customer.c */
#include "shm_com.h"
#include "sem_com.h"

int main()
{
    void *shared_memory = NULL;
    struct shm_buff *shm_buff_inst;
    int shmid, semid;
    /* 获得信号量 */
    semid = semget(ftok(".", 'a'), 1, 0666);
    if (semid == -1)
    {
        perror("Producer is't exist");
        exit(1);
    }
    /* 获得共享内存 */
    shmid = shmget(ftok(".", 'b'), sizeof(struct shm_buff), 0666|IPC_CREAT);
    if (shmid == -1)
    {
        perror("shmget");
        exit(1);
    }
    /* 将共享内存地址映射到当前进程地址空间 */
    shared_memory = shmat(shmid, (void*)0, 0);
    if (shared_memory == (void*)-1)

```

```

{
    perror("shmat");
    exit(1);
}

printf("Memory attached at %X\n", (int)shared_memory);
/* 获得共享内存的映射地址 */
shm_buff_inst = (struct shm_buff *)shared_memory;
do
{
    sem_p(semid);
    printf("Shared memory was written by process %d :%s"
        , shm_buff_inst->pid, shm_buff_inst->buffer);
    if (strcmp(shm_buff_inst->buffer, "quit", 4) == 0)
    {
        break;
    }
    shm_buff_inst->pid = 0;
    memset(shm_buff_inst->buffer, 0, SHM_BUFF_SZ);
    sem_v(semid);
} while(1);

/* 删除共享内存到当前进程地址空间中的映射 */
if (shmdt(shared_memory) == -1)
{
    perror("shmdt");
    exit(1);
}

/* 删除共享内存 */
if (shmctl(shmid, IPC_RMID, NULL) == -1)
{
    perror("shmctl(IPC_RMID)");
    exit(1);
}

exit(0);
}

```

14、消息队列的实现过程：p109 下部

4.6 消息队列

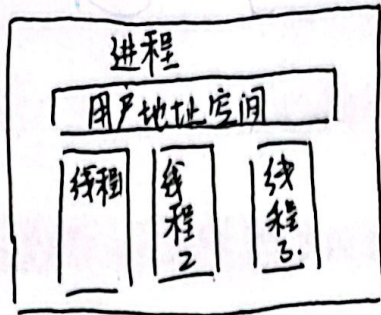
顾名思义，消息队列就是一些消息的列表。用户可以在消息队列中添加消息和读取消息等。从这点上看，消息队列具有一定的 FIFO 特性，但是它可以实现消息的随机查询，比 FIFO 具有更大的优势。同时，这些消息又存在于内核中，由“队列 ID”来标识。

△消息队列的实现包括创建或打开消息队列、添加消息、读取消息和控制消息队列这 4 种操作。其中创建或打开消息队列使用的函数是 `msgget()`，这里创建的消息队列的数量会受到系统消息队列数量的限制；添加消息使用的函数是 `msgsnd()`，它把消息添加到已打开的消息队列末尾；读取消息使用的函数是 `msgrcv()`，它把消息从消息队列中取走，与 FIFO 不同的是，这里可

以取走指定的某一条消息；最后，控制消息队列使用的函数是 `msgctl()`，它可以完成多项功能。

五.多线程技术.

1. 线程运行在用户空间.



2. 线程的几个函数.

(1) `pthread_create()` 创建线程
成功返回"0"

(2) `pthread_join()` 阻塞等待线程结束
成功返回"0" (线程阻塞函数, 线程结束时, 资源被收回)

(3) `pthread_exit()` 退出线程(主动)
注: `pthread_exit()`: 终止所有线程

(4) `pthread_cancel()` 取消线程

3. 线程之间的同步与互斥 — 互斥锁和信号量

(1) 互斥锁 (通过加锁保证资源共享操作进行)

两种状态 { 上锁
解锁

↓
按顺序

在同一时刻只能有一个线程有某个互斥锁, 且该线程能对共享资源进行操作

函数

(i) 初始化: `pthread_mutex_init()`

(ii) 上锁: `pthread_mutex_lock()` // 不成功则阻塞加锁

(iii) 判断上锁: `pthread_mutex_trylock()` // 不成功则返回

(iv) 解锁: `pthread_mutex_unlock()`

(v) 消除: `pthread_mutex_destroy()`



12) 信号量 (P/V 操作)

· 函数

`sem_init()` - 初始化

`sem_wait()` - P 操作 (-1), 阻塞

`sem_trywait()` - 立即返回

`sem_post()` - V 操作 (+1) 唤醒线程

4. 信号量的同步与互斥原理:

p135 图 5-1, 5-2

p136-138 代码

5. 线程的属性包括哪些内容?

分离属性: 决定一个线程何时释放自己的资源
堆栈大小
调度策略
优先级

分离: 线程结束, 立即释放资源
非分离: 线程结束, 资源未释放, 未结束

默认属性: 非分离, 默认 1MB 堆栈, 与主线程同样调度策略, 相同的优先级

6. 生产者与消费者 2 个线程进行同步与互斥的示例

(1) 为何 3 个信号量?

`avail` 和 `full` 分别用于解决生产者和消费者线程间的同步问题, `mutex` 用于这两个线程之间的互斥问题.

7. 线程通信的原理图及代码: p144-150

六、网络通信

1、OSI与TCP/IP的参考模型分别是几层？对应的关系？ 2、每层的常见协议是什么？ p155

OSI有7层 TCP/IP有4层

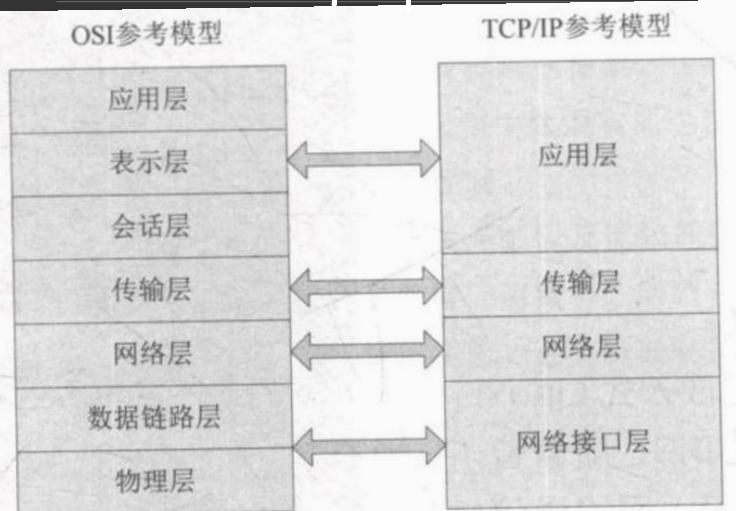


图 6.1 OSI 参考模型和 TCP/IP 参考模型对应关系



图 6.3 TCP/IP 协议族不同分层中的协议

3、TCP与UDP的区别与联系？ p167中部 *代码自看*

一、联系

TCP/IP 是互联网相关的各类协议族的总称，比如：TCP，UDP，IP，FTP，HTTP，ICMP，SMTP 等都属于 TCP/IP 族内的协议。

二、区别

- 1、TCP面向连接，UDP是无连接的，即发送数据之前不需要建立连接。
- 2、TCP提供可靠的服务，UDP则不提供。
- 3、TCP面向字节流，而UDP是面向报文的，而且没有拥塞控制。
- 4、每一条TCP连接只能是点到点的；UDP支持一对一，一对多，多对一和多对多的交互通信。
- 5、TCP的逻辑通信信道是全双工的可靠信道，UDP则是不可靠信道，但是传输速度快。

4、Socket的类型？

常见的 Socket 类型有如下 3 种。

(1) 流式套接字 (SOCK_STREAM)。

流式套接字提供可靠的、面向连接的通信流，保证数据传输的可靠性和按序收发。TCP 通信使用的就是流式套接字。

(2) 数据报套接字 (SOCK_DGRAM)。

数据报套接字实现了一种不可靠、无连接的服务。数据通过相互独立的报文进行传输，是无序的，并且不保证可靠的传输。UDP 通信使用的就是数据报套接字。

(3) 原始套接字 (SOCK_RAW)。

原始套接字允许对底层协议（如 IP 或 ICMP）进行直接访问，它功能强大但使用较为不便，主要用于一些协议的开发。

5、TCP Socket与UDP Socket的流程描述？

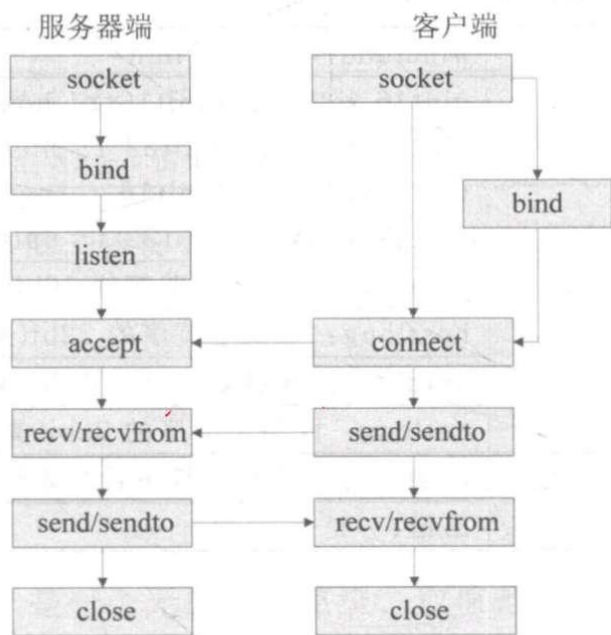


图 6.7 使用 TCP 时 Socket 编程流程图

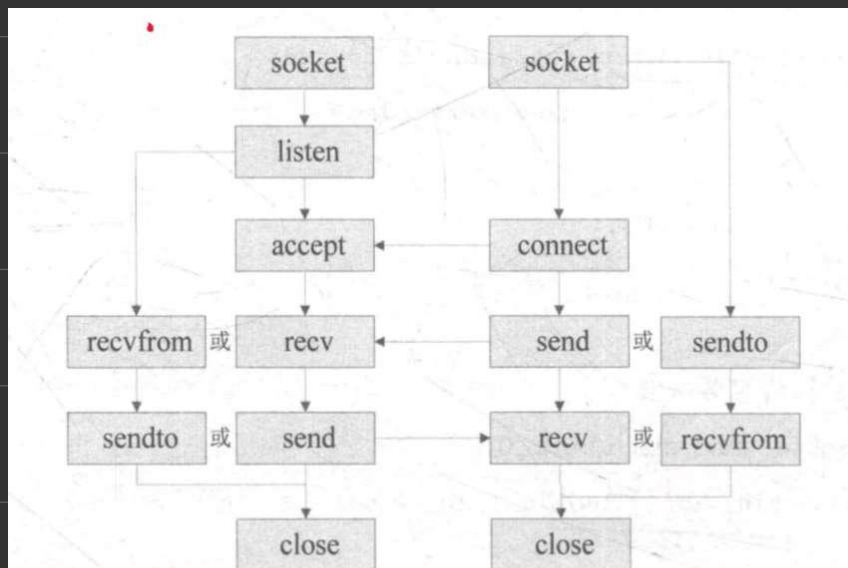


图 6.8 使用 UDP 时 Socket 编程流程图